



**UNIVERSIDAD
DON BOSCO**

DESARROLLO DE SOFTWARE PARA MOVILES DSM941 VIRTUAL

SEGUNDO PROYECTO KOTLIN:

Aplicación de Gestión de Eventos Comunitarios

ENLACE A REPOSITORIO EN GITHUB:

AQUÍ

Frank Alberto Hernández Silva HS171707

Andrea Marcela Rico Figueroa RF160050

Marcos Daniel Ibáñez Guevara IG243224

San Salvador, 24 de mayo de 2026

APLICACIÓN DE GESTIÓN DE EVENTOS COMUNITARIOS

Parte 1: Estructura de Datos y Persistencia

1. build.gradle.kts (Module :app)

- **¿Qué contiene?** Es el archivo de configuración de dependencias y librerías del módulo principal del proyecto.
- **¿Qué hace?** Le indica a Android Studio qué herramientas externas debe descargar e inyectar. Aquí incluimos las librerías nativas de **Jetpack Navigation Component** (navigation-fragment-ktx y navigation-ui-ktx). Sin ellas, el compilador no reconocería componentes críticos como el NavHostFragment o el controlador de rutas (NavController).

```
// Componentes de navegación tradicionales (Fragments)
implementation("androidx.navigation:navigation-fragment-ktx:2.7.7")
implementation("androidx.navigation:navigation-ui-ktx:2.7.7")
implementation("com.google.android.material:material:1.11.0")

implementation("androidx.core:core-ktx:1.12.0")
implementation("androidx.appcompat:appcompat:1.6.1")
implementation("androidx.constraintlayout:constraintlayout:2.1.4")
```

2. DatabaseHelper.kt

- **¿Qué contiene?** Contiene la lógica estructural de **SQLite**, un motor de base de datos relacional ligero integrado directamente en el sistema operativo de tu teléfono.
- **¿Qué hace?**
 - **Creación:** Al ejecutarse por primera vez, crea de forma autónoma 4 tablas relacionales mediante sentencias SQL: usuarios (credenciales cifradas localmente), eventos (detalles de la cartelera), comentarios (opiniones y estrellas) e historial (vínculo entre qué usuario asiste a qué evento).
 - **Semilla (Seed):** Inyecta automáticamente dos eventos comunitarios reales de prueba (*Torneo de Atletismo UDB* y *Feria de Software y Robótica*) para que la aplicación nunca aparezca vacía al iniciar.

```
package com.example.navegacionapp

import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DatabaseHelper(context: Context) {
```

```

SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION)
{

    companion object {
        private const val DATABASE_NAME = "ComunidadEventos.db"
        private const val DATABASE_VERSION = 1

        const val TABLA_USUARIOS = "usuarios"
        const val KEY_USER_ID = "id"
        const val KEY_USER_EMAIL = "email"
        const val KEY_USER_PASS = "password"

        const val TABLA_EVENTOS = "eventos"
        const val KEY_EV_ID = "id"
        const val KEY_EV_TITULO = "titulo"
        const val KEY_EV_DESC = "descripcion"
        const val KEY_EV_FECHA = "fecha"
        const val KEY_EV_HORA = "hora"
        const val KEY_EV_UBICACION = "ubicacion"

        const val TABLA_COMENTARIOS = "comentarios"
        const val KEY_COM_ID = "id"
        const val KEY_COM_EVENTO_ID = "evento_id"
        const val KEY_COM_USUARIO = "usuario"
        const val KEY_COM_TEXTO = "texto"
        const val KEY_COM_ESTRELLAS = "estrellas"

        const val TABLA_HISTORIAL = "historial"
        const val KEY_HIST_ID = "id"
        const val KEY_HIST_USER_EMAIL = "user_email"
        const val KEY_HIST_EVENTO_ID = "evento_id"
    }

    override fun onCreate(db: SQLiteDatabase) {
        val crearUsuarios = ("CREATE TABLE " + TABLA_USUARIOS +
            "("
                + KEY_USER_ID + " INTEGER PRIMARY KEY
            AUTOINCREMENT,"
                + KEY_USER_EMAIL + " TEXT UNIQUE,"
                + KEY_USER_PASS + " TEXT)")

        val crearEventos = ("CREATE TABLE " + TABLA_EVENTOS +
            "("
                + KEY_EV_ID + " INTEGER PRIMARY KEY
            AUTOINCREMENT,"
                + KEY_EV_TITULO + " TEXT,"
                + KEY_EV_DESC + " TEXT,"
                + KEY_EV_FECHA + " TEXT,"
                + KEY_EV_HORA + " TEXT,"
                + KEY_EV_UBICACION + " TEXT)")

        val crearComentarios = ("CREATE TABLE " +
            TABLA_COMENTARIOS + "("
                + KEY_COM_ID + " INTEGER PRIMARY KEY
            AUTOINCREMENT,"
                + KEY_COM_EVENTO_ID + " INTEGER,"
                + KEY_COM_USUARIO + " TEXT,"
                + KEY_COM_TEXTO + " TEXT,"
                + KEY_COM_ESTRELLAS + " INTEGER)")

        val crearHistorial = ("CREATE TABLE " + TABLA_HISTORIAL

```

```

+ "("
+ KEY_HIST_ID + " INTEGER PRIMARY KEY
AUTOINCREMENT,"
+ KEY_HIST_USER_EMAIL + " TEXT,"
+ KEY_HIST_EVENTO_ID + " INTEGER)"")

db.execSQL(crearUsuarios)
db.execSQL(crearEventos)
db.execSQL(crearComentarios)
db.execSQL(crearHistorial)

// Datos semilla precargados automáticos para la defensa
db.execSQL("INSERT INTO $TABLA_EVENTOS ($KEY_EV_TITULO,
$KEY_EV_DESC, $KEY_EV_FECHA, $KEY_EV_HORA, $KEY_EV_UBICACION)
VALUES ('Torneo de Atletismo UDB', 'Competencia de saltos y
velocidad académica.', '2026-06-15', '08:00', 'Polideportivo
UDB')")

db.execSQL("INSERT INTO $TABLA_EVENTOS ($KEY_EV_TITULO,
$KEY_EV_DESC, $KEY_EV_FECHA, $KEY_EV_HORA, $KEY_EV_UBICACION)
VALUES ('Feria de Software y Robótica', 'Exposición de proyectos
de fin de ciclo.', '2026-05-30', '10:00', 'Gimnasio
Universitario')")
}

override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int,
newVersion: Int) {
    db.execSQL("DROP TABLE IF EXISTS $TABLA_USUARIOS")
    db.execSQL("DROP TABLE IF EXISTS $TABLA_EVENTOS")
    db.execSQL("DROP TABLE IF EXISTS $TABLA_COMENTARIOS")
    db.execSQL("DROP TABLE IF EXISTS $TABLA_HISTORIAL")
    onCreate(db)
}
}

```

Parte 2: El Módulo de Autenticación (Requisito 1)

3. activity_login.xml

- **¿Qué contiene?** Es la interfaz visual (UI) de la pantalla de acceso, diseñada con una estructura de contenedor vertical (LinearLayout).
- **¿Qué hace?** Provee los campos de captura de datos en pantalla: dos cajas de texto especializadas (EditText) para el correo electrónico y la contraseña con enmascaramiento de caracteres de seguridad, además de los botones de interacción para validar la sesión o registrar una cuenta nueva.

```

• <?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp"
    android:gravity="center"
    android:background="#F5F5F5">

    <TextView
        android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        android:text="Eventos Comunitarios UDB"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="#0D47A1"
        android:layout_marginBottom="32dp"/>

        <EditText
            android:id="@+id/edtEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Correo Electrónico"
            android:inputType="textEmailAddress"
            android:layout_marginBottom="16dp"/>

        <EditText
            android:id="@+id/edtPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Contraseña"
            android:inputType="textPassword"
            android:layout_marginBottom="24dp"/>

        <Button
            android:id="@+id/btnLogin"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Iniciar Sesión"
            android:backgroundTint="#0D47A1"/>

        <Button
            android:id="@+id/btnRegister"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Registrarse"

            style="@style/Widget.MaterialComponents.Button.TextButton"/>
    </LinearLayout>

```

4. LoginActivity.kt

- ¿Qué contiene? Es el controlador lógico que gobierna la pantalla de Login.
- ¿Qué hace?
 - **Registro:** Si el usuario presiona "Registrarse", toma los textos, los empaqueta en un contenedor de valores (ContentValues) y ejecuta un comando db.insert en la tabla de usuarios de SQLite.
 - **Inicio de Sesión:** Si presiona "Iniciar Sesión", ejecuta un cursor con una consulta selectiva (SELECT * FROM usuarios WHERE email=? AND password=?). Si hay coincidencia, guarda el correo de forma persistente en las preferencias del teléfono (SharedPreferences) bajo la clave "SesionUsuario" para saber quién está interactuando, abre la actividad principal (MainActivity) y destruye la pantalla de login con un finish() para que el usuario no pueda regresar a ella al presionar el botón "Atrás".

```

• package com.example.navegacionapp

import android.content.ContentValues
import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class LoginActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_login)

        val dbHelper = DatabaseHelper(this)
        val edtEmail = findViewById<EditText>(R.id.edtEmail)
        val edtPassword =
            findViewById<EditText>(R.id.edtPassword)
        val btnLogin = findViewById<Button>(R.id.btnLogin)
        val btnRegister = findViewById<Button>(R.id.btnRegister)

        btnLogin.setOnClickListener {
            val email = edtEmail.text.toString().trim()
            val pass = edtPassword.text.toString().trim()

            if (email.isEmpty() || pass.isEmpty()) {
                Toast.makeText(this, "Por favor llena todos los campos", Toast.LENGTH_SHORT).show()
            } else {
                val db = dbHelper.readableDatabase
                val cursor = db.rawQuery(
                    "SELECT * FROM ${DatabaseHelper.TABLA_USUARIOS} WHERE ${DatabaseHelper.KEY_USER_EMAIL}=? AND ${DatabaseHelper.KEY_USER_PASS}=?",
                    arrayOf(email, pass)
                )
                if (cursor.moveToFirst()) {
                    val prefs =
                        getSharedPreferences("SesionUsuario", Context.MODE_PRIVATE)
                    prefs.edit().putString("email", email).apply()

                    startActivity(Intent(this, MainActivity::class.java))
                    finish()
                } else {
                    Toast.makeText(this, "Credenciales incorrectas", Toast.LENGTH_SHORT).show()
                }
                cursor.close()
            }
        }

        btnRegister.setOnClickListener {
            val email = edtEmail.text.toString().trim()
            val pass = edtPassword.text.toString().trim()

```

```

        if (email.isEmpty() || pass.isEmpty()) {
            Toast.makeText(this, "Llena los campos para
registrarte", Toast.LENGTH_SHORT).show()
        } else {
            val db = dbHelper.writableDatabase
            val values = ContentValues().apply {
                put(DatabaseHelper.KEY_USER_EMAIL, email)
                put(DatabaseHelper.KEY_USER_PASS, pass)
            }
            val resultado =
db.insert(DatabaseHelper.TABLA_USUARIOS, null, values)
            if (resultado != -1L) {
                Toast.makeText(this, "Usuario registrado con
éxito", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "El usuario ya existe o
hubo un error", Toast.LENGTH_SHORT).show()
            }
        }
    }
}
}
}

```

Parte 3: Contenedor Principal (Navegación Unificada)

5. navigation_menu.xml

- **¿Qué contiene?** Un archivo de recursos tipo menú que agrupa los destinos de la aplicación.
- **¿Qué hace?** Define los títulos (Inicio, Galería / Historial, Presentación) y les asigna íconos seguros del sistema (@android:drawable/...). Su regla fundamental es que **el ID de cada ítem de menú coincide exactamente con el ID asignado en el grafo de navegación**. Esto permite que la librería de Android realice la transición de pantallas de forma automática sin escribir código manual.

```

• <?xml version="1.0" encoding="utf-8"?>
  <menu
    xmlns:android="http://schemas.android.com/apk/res/android">
    <group android:checkableBehavior="single">
      <item
        android:id="@+id/fragmentHome"
        android:icon="@android:drawable/ic_dialog_info"
        android:title="Inicio" />
      <item
        android:id="@+id/fragmentGallery"
        android:icon="@android:drawable/ic_menu_manage"
        android:title="Galería / Historial" />
      <item
        android:id="@+id/fragmentSlideshow"
        android:icon="@android:drawable/ic_menu_search"
        android:title="Presentación" />
    </group>
  </menu>

```

6. nav_graph.xml

- **¿Qué contiene?** El **Grafo de Navegación** en formato XML. Es el mapa de carreteras del proyecto.
- **¿Qué hace?** Declara de forma estática qué fragmentos existen en la aplicación (HomeFragment, GalleryFragment, SlideshowFragment), cuál es la pantalla inicial por defecto (app:startDestination) y qué etiqueta de título llevará la barra superior al estar activos.

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/fragmentHome">

    <fragment
        android:id="@+id/fragmentHome"
        android:name="com.example.navegacionapp.HomeFragment"
        android:label="Gestión de Eventos"
        tools:layout="@layout/fragment_home" />

    <fragment
        android:id="@+id/fragmentGallery"
        android:name="com.example.navegacionapp.GalleryFragment"
        android:label="Mi Historial"
        tools:layout="@layout/fragment_gallery" />

    <fragment
        android:id="@+id/fragmentSlideshow"
        android:name="com.example.navegacionapp.SlideshowFragment"
        android:label="Presentación"
        tools:layout="@layout/fragment_slideshow" />
</navigation>
```

7. activity_main.xml

- **¿Qué contiene?** El cascarón visual principal que fusiona los componentes de Material Design.
- **¿Qué hace?** Organiza el espacio en pantalla:
 1. **DrawerLayout:** Permite que el menú lateral "hamburguesa" se deslice desde el borde izquierdo.
 2. **Toolbar:** La barra superior que reemplaza la barra del sistema para mostrar el título dinámico.
 3. **NavHostFragment:** El área en blanco donde se inyectan y destruyen los fragmentos según navegues.
 4. **BottomNavigationView & NavigationView:** La barra inferior y el panel lateral que leen los datos de navigation_menu.xml.


```

• <?xml version="1.0" encoding="utf-8"?>
  <androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true">

    <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:orientation="vertical">

      <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorPrimary"

        android:theme="@style/ThemeOverlay.AppCompat.Dark.ActionBar"
        app:popupTheme="@style/ThemeOverlay.AppCompat.Light"
      />

      <fragment
        android:id="@+id/nav_host_fragment"

        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph" />

      <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="?android:attr/windowBackground"
        app:menu="@menu/navigation_menu" />
      </LinearLayout>

      <com.google.android.material.navigation.NavigationView
        android:id="@+id/navigation_view"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        android:fitsSystemWindows="true"
        app:menu="@menu/navigation_menu" />

    </androidx.drawerlayout.widget.DrawerLayout>

```

8. MainActivity.kt

- **¿Qué contiene?** La lógica central de sincronización de la arquitectura de la app.
- **¿Qué hace?** Utiliza la potente API `setupWithNavController`. Conecta el controlador de rutas (`NavController`) tanto con el menú lateral como con la barra inferior. Al acoplarlos con un `AppBarConfiguration`, la actividad sabe cuándo debe mostrar las tres líneas del menú "hamburguesa" y cuándo sincronizar los fragmentos de forma unificada.

```
package com.example.navegacionapp

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.drawerlayout.widget.DrawerLayout
import androidx.navigation.NavController
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.AppBarConfiguration
import androidx.navigation.ui.setupWithNavController
import androidx.navigation.ui.setupActionBarWithNavController
import androidx.navigation.ui.NavigationUI
import com.google.android.material.bottomnavigation.BottomNavigationView

import com.google.android.material.navigation.NavigationView
import androidx.appcompat.widget.Toolbar

class MainActivity : AppCompatActivity() {

    private lateinit var navController: NavController
    private lateinit var drawerLayout: DrawerLayout
    private lateinit var appBarConfiguration:
AppBarConfiguration

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar: Toolbar = findViewById(R.id.toolbar)
        setSupportActionBar(toolbar)

        drawerLayout = findViewById(R.id.drawer_layout)
        val navView: NavigationView =
findViewById(R.id.navigation_view)
        val bottomNav: BottomNavigationView =
findViewById(R.id.bottom_navigation)

        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as
NavHostFragment
        navController = navHostFragment.navController

        appBarConfiguration = AppBarConfiguration(
            setOf(R.id.fragmentHome, R.id.fragmentGallery,
R.id.fragmentSlideshow),
            drawerLayout
        )

        setupActionBarWithNavController(navController,
appBarConfiguration)
    }
}
```

```

        navView.setupWithNavController(navController)
        bottomNav.setupWithNavController(navController)
    }

    override fun onSupportNavigateUp(): Boolean {
        return NavigationUI.navigateUp(navController,
            appBarConfiguration) || super.onSupportNavigateUp()
    }
}

```

Parte 4: Gestión de Eventos y Área Social (Requisitos 2 y 3)

9. fragment_home.xml

- **¿Qué contiene?** La interfaz visual de la pantalla de inicio protegida dentro de un contenedor de desplazamiento (ScrollView) para evitar que los elementos se corten en pantallas pequeñas.
- **¿Qué hace?** Divide la pantalla en tres zonas operativas: la zona superior tiene el formulario CRUD para escribir y publicar nuevos eventos; la zona media muestra mediante un bloque dinámico de texto el último evento registrado en cartelera con su botón de asistencia; y la zona inferior expone la caja de comentarios con su sistema numérico de calificación por estrellas.

```

• <?xml version="1.0" encoding="utf-8"?>
  <ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF">

    <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:orientation="vertical"
      android:padding="16dp">

      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Publicar Nuevo Evento Comunitario"
        android:textStyle="bold"
        android:textColor="#0D47A1"
        android:textSize="16sp"
        android:layout_marginBottom="8dp"/>

      <EditText
        android:id="@+id/edtTituloEv"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Título del Evento"/>

      <EditText
        android:id="@+id/edtUbicacionEv"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Ubicación / Lugar"/>
    </LinearLayout>
  </ScrollView>

```

```

<Button
    android:id="@+id/btnGuardarEv"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Registrar Evento"
    android:backgroundTint="#1B5E20"
    android:layout_marginBottom="24dp"/>

<View
    android:layout_width="match_parent"
    android:layout_height="2dp"
    android:background="#EEEEEE"
    android:layout_marginBottom="16dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Próximo Evento en Cartelera"
    android:textStyle="bold"
    android:textColor="#0D47A1"
    android:textSize="16sp"
    android:layout_marginBottom="8dp"/>

<TextView
    android:id="@+id/txtDetalleEvento"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Cargando evento..."
    android:textSize="15sp"
    android:lineSpacingMultiplier="1.2"
    android:layout_marginBottom="12dp"/>

<Button
    android:id="@+id/btnAsistir"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Marcar Asistencia"
    android:backgroundTint="#E65100"
    android:layout_marginBottom="16dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Sección de Comentarios y Valoración"
    android:textStyle="bold"
    android:textSize="14sp"
    android:layout_marginBottom="4dp"/>

<EditText
    android:id="@+id/edtComentario"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Escribe tu opinión del evento..."/>

<EditText
    android:id="@+id/edtEstrellas"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Calificación (1 a 5 estrellas)"
    android:inputType="number"/>

```

```

        <Button
            android:id="@+id/btnEnviarComentario"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Publicar Comentario"
            android:layout_marginBottom="16dp"/>

        <TextView
            android:id="@+id/txtListaComentarios"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="No hay comentarios aún."
            android:textColor="#666666"
            android:textSize="13sp"/>
    </LinearLayout>
</ScrollView>

```

10. HomeFragment.kt

- ¿Qué contiene? La lógica interactiva de la pantalla de inicio del proyecto.
- ¿Qué hace?
 - **Publicar (CRUD):** Lee los campos de texto y realiza un db.insert en la tabla eventos, refrescando la pantalla al instante.
 - **Interacción Social:** Al presionar "Publicar Comentario", extrae el correo del usuario guardado en las preferencias (SharedPreferences), lee el texto y el número de estrellas, y los añade a la tabla comentarios.
 - **Consulta Dinámica:** Contiene la función interna cargarUltimoEvento(), la cual ejecuta una consulta ordenada (SELECT * FROM eventos ORDER BY id DESC LIMIT 1) para renderizar en pantalla los detalles actualizados del evento comunitario junto a la lista unificada de todos los comentarios de la comunidad que le pertenecen.

```

• package com.example.navegacionapp

import android.content.ContentValues
import android.content.Context
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.*
import androidx.fragment.app.Fragment

class HomeFragment : Fragment() {

    private var eventoActualId: Int = 1

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_home,
            container, false)
    }
}

```

```

        val dbHelper = DatabaseHelper(requireContext())
        val prefs =
            requireActivity().getSharedPreferences("SesionUsuario",
                Context.MODE_PRIVATE)
        val usuarioLogueado = prefs.getString("email",
            "Anónimo") ?: "Anónimo"

        val edtTitulo =
            view.findViewById<EditText>(R.id.edtTituloEv)
        val edtUbicacion =
            view.findViewById<EditText>(R.id.edtUbicacionEv)
        val btnGuardarEv =
            view.findViewById<Button>(R.id.btnGuardarEv)
        val txtDetalle =
            view.findViewById<TextView>(R.id.txtDetalleEvento)
        val btnAsistir =
            view.findViewById<Button>(R.id.btnAsistir)

        val edtComentario =
            view.findViewById<EditText>(R.id.edtComentario)
        val edtEstrellas =
            view.findViewById<EditText>(R.id.edtEstrellas)
        val btnEnviarComentario =
            view.findViewById<Button>(R.id.btnEnviarComentario)
        val txtListaComentarios =
            view.findViewById<TextView>(R.id.txtListaComentarios)

        fun cargarUltimoEvento() {
            val db = dbHelper.readableDatabase
            val cursor = db.rawQuery("SELECT * FROM
                ${DatabaseHelper.TABLA_EVENTOS} ORDER BY
                ${DatabaseHelper.KEY_EV_ID} DESC LIMIT 1", null)
            if (cursor.moveToFirst()) {
                eventoActualId =
                    cursor.getInt(cursor.getColumnIndexOrThrow(DatabaseHelper.KEY_EV_ID))

                val titulo =
                    cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.KEY_EV_TITULO))
                val ubicacion =
                    cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.KEY_EV_UBICACION))
                val fecha =
                    cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.KEY_EV_FECHA))

                txtDetalle.text = "📌 Título: $titulo\n📍 Lugar: $ubicacion\n📅 Fecha: $fecha"
            }
            cursor.close()

            val cursorCom = db.rawQuery("SELECT * FROM
                ${DatabaseHelper.TABLA_COMENTARIOS} WHERE
                ${DatabaseHelper.KEY_COM_EVENTO_ID} = ?",
                arrayOf(eventoActualId.toString()))
            var cadenaComentarios = "Comentarios de la Comunidad:\n"
            if (cursorCom.moveToFirst()) {
                do {
                    val user =

```

```

cursorCom.getString(cursorCom.getColumnIndexOrThrow(DatabaseHelp
er.KEY_COM_USUARIO))
        val text =
cursorCom.getString(cursorCom.getColumnIndexOrThrow(DatabaseHelp
er.KEY_COM_TEXTO))
        val stars =
cursorCom.getInt(cursorCom.getColumnIndexOrThrow(DatabaseHelper.
KEY_COM_ESTRELLAS))
        cadenaComentarios += "👤 $user (${stars}★):
$text\n"
    } while (cursorCom.moveToNext())
    txtListaComentarios.text = cadenaComentarios
} else {
    txtListaComentarios.text = "No hay valoraciones
en este momento."
}
cursorCom.close()
}

btnGuardarEv.setOnClickListener {
    val t = edtTitulo.text.toString().trim()
    val u = edtUbicacion.text.toString().trim()
    if(t.isNotEmpty() && u.isNotEmpty()) {
        val db = dbHelper.writableDatabase
        val v = ContentValues().apply {
            put(DatabaseHelper.KEY_EV_TITULO, t)
            put(DatabaseHelper.KEY_EV_UBICACION, u)
            put(DatabaseHelper.KEY_EV_FECHA, "2026-06-
01")
            put(DatabaseHelper.KEY_EV_HORA, "14:00")
        }
        db.insert(DatabaseHelper.TABLA_EVENTOS, null, v)
        Toast.makeText(context, "¡Evento publicado!",
Toast.LENGTH_SHORT).show()
        edtTitulo.text.clear()
        edtUbicacion.text.clear()
        cargarUltimoEvento()
    }
}

btnAsistir.setOnClickListener {
    val db = dbHelper.writableDatabase
    val v = ContentValues().apply {
        put(DatabaseHelper.KEY_HIST_USER_EMAIL,
usuarioLogueado)
        put(DatabaseHelper.KEY_HIST_EVENTO_ID,
eventoActualId)
    }
    db.insert(DatabaseHelper.TABLA_HISTORIAL, null, v)
    Toast.makeText(context, "Asistencia guardada en el
historial", Toast.LENGTH_SHORT).show()
}

btnEnviarComentario.setOnClickListener {
    val comText = edtComentario.text.toString().trim()
    val starVal = edtEstrellas.text.toString().trim()
    if (comText.isNotEmpty() && starVal.isNotEmpty()) {
        val db = dbHelper.writableDatabase
        val v = ContentValues().apply {
            put(DatabaseHelper.KEY_COM_EVENTO_ID,
eventoActualId)

```

```

        usuarioLogueado)
        put(DatabaseHelper.KEY_COM_USUARIO,
        put(DatabaseHelper.KEY_COM_TEXTO, comText)
        put(DatabaseHelper.KEY_COM_ESTRELLAS,
starVal.toInt())
    }
    db.insert(DatabaseHelper.TABLA_COMENTARIOS,
null, v)
    edtComentario.text.clear()
    edtEstrellas.text.clear()
    cargarUltimoEvento()
    }
    }
    cargarUltimoEvento()
    return view
}
}

```

Parte 5: Historial y Presentación (Requisito 4)

11. fragment_gallery.xml

- **¿Qué contiene?** La interfaz visual de la bitácora de control de usuario.
- **¿Qué hace?** Contiene un contenedor con fondo personalizado que aloja un componente de texto grande (TextView) diseñado específicamente para actuar como lienzo dinámico donde se listarán las asistencias del estudiante.

```

• <?xml version="1.0" encoding="utf-8"?>
  <LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="#F0F4C3">

    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Mi Historial de Asistencia"
      android:textSize="18sp"
      android:textStyle="bold"
      android:textColor="#33691E"
      android:layout_marginBottom="16dp"/>

    <TextView
      android:id="@+id/txtHistorial"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="No posees registros de asistencia en tu
cuenta."
      android:textSize="15sp"
      android:lineSpacingMultiplier="1.3"
      android:textColor="#2E7D32"/>
  </LinearLayout>

```


12. GalleryFragment.kt

- ¿Qué contiene? El controlador de lógica del historial del usuario activo.
- ¿Qué hace? Resuelve el requerimiento técnico más complejo del laboratorio: ejecuta una consulta relacional avanzada de base de datos conocida como **INNER JOIN**. Consulta la tabla de historial buscando el correo de la sesión activa, cruza los datos con la tabla de eventos usando el ID como puente, y concatena mediante un bucle de control (do-while) una lista ordenada con viñetas que muestra el título exacto y la fecha de cada evento donde el usuario presionó el botón "Marcar Asistencia".

```
package com.example.navegacionapp

import android.content.Context
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.fragment.app.Fragment

class GalleryFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_gallery,
            container, false)
        val txtHistorial =
            view.findViewById<TextView>(R.id.txtHistorial)

        val dbHelper = DatabaseHelper(requireContext())
        val prefs =
            requireActivity().getSharedPreferences("SesionUsuario",
                Context.MODE_PRIVATE)
        val usuarioLogueado = prefs.getString("email",
            "Anónimo") ?: "Anónimo"

        val db = dbHelper.readableDatabase

        val query = """
            SELECT e.${DatabaseHelper.KEY_EV_TITULO},
            e.${DatabaseHelper.KEY_EV_FECHA}
            FROM ${DatabaseHelper.TABLA_HISTORIAL} h
            INNER JOIN ${DatabaseHelper.TABLA_EVENTOS} e ON
            h.${DatabaseHelper.KEY_HIST_EVENTO_ID} =
            e.${DatabaseHelper.KEY_EV_ID}
            WHERE h.${DatabaseHelper.KEY_HIST_USER_EMAIL} = ?
        """.trimIndent()

        val cursor = db.rawQuery(query,
            arrayOf(usuarioLogueado))

        if (cursor.moveToFirst()) {
            var historialTexto = "Eventos a los que te has
            inscrito:\n\n"
            var contador = 1
```

```

        do {
            val titulo = cursor.getString(0)
            val fecha = cursor.getString(1)
            historialTexto += "$contador. ☒ $titulo (Fecha:
$fecha)\n"
            contador++
        } while (cursor.moveToNext())
        txtHistorial.text = historialTexto
    } else {
        txtHistorial.text = "Hola $usuarioLogueado,\nNo hay
asistencias marcadas aún."
    }
    cursor.close()

    return view
}
}

```

13. fragment_slideshow.xml

- **¿Qué contiene?** La interfaz visual de la pantalla de créditos académicos y de software.
- **¿Qué hace?** Muestra de forma elegante y centrada mediante un contenedor de alineación (FrameLayout) los títulos informativos del laboratorio, los datos formales de la asignatura y, de forma obligatoria, la declaración explícita de la **Licencia Creative Commons (CC BY-NC-SA 4.0)** para proteger los derechos de autoría de la solución frente a explotación comercial.

```

• <?xml version="1.0" encoding="utf-8"?>
  <FrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFF3E0"
    android:padding="16dp">

    <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_gravity="center"
      android:orientation="vertical"
      android:gravity="center">

      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Información del Proyecto"
        android:textColor="#E65100"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_marginBottom="16dp"/>

      <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Gestión de Eventos Comunitarios
v1.0\nDesarrollo de Software para Móviles"

```

```

        android:textColor="#555555"
        android:textSize="15sp"
        android:gravity="center"
        android:layout_marginBottom="24dp"/>

        <View
            android:layout_width="100dp"
            android:layout_height="1dp"
            android:background="#E65100"
            android:layout_marginBottom="24dp"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Licencia de Software Aplicada:"
            android:textColor="#333333"
            android:textSize="14sp"
            android:textStyle="bold"
            android:layout_marginBottom="8dp"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Creative Commons\nAtribución-
NoComercial-CompartirIgual\n(CC BY-NC-SA 4.0)"
            android:gravity="center"
            android:textColor="#E65100"
            android:textSize="14sp"
            android:textStyle="italic" />
    </LinearLayout>
</FrameLayout>

```

14. SlideshowFragment.kt

- **¿Qué contiene?** El archivo de control básico del fragmento de presentación.
- **¿Qué hace?** Cumple con el estándar de ciclo de vida de los fragmentos en Android. Llama al método `onCreateView` para inflar el recurso visual de la licencia en pantalla cuando el usuario selecciona esta opción en cualquiera de los menús unificados de la aplicación.

```

package com.example.navegacionapp

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class SlideshowFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Enlaza de forma síncrona el código Kotlin con el
        diseño visual XML
        return inflater.inflate(R.layout.fragment_slideshow,

```

```
container, false)  
    }  
}
```



**UNIVERSIDAD
DON BOSCO**